Kruskal's Minimum Network: Solution

Emirates Post Network Audit · Design and Analysis of Algorithms · Network Analyst

REFERENCE KEY for checking your own paper trace. The brief requires the hand-trace to be written on paper by the person named on it: copy the method into your own handwriting, work it from the sorted list, and use this sheet only to check your answer.

1. Problem

The Data Lead's threshold (X = 38 min) leaves 13 van routes between 7 post offices, 334 min of driving in total. Running every route is the most expensive option. Kruskal's algorithm finds the cheapest set of routes that still lets a parcel reach every office.

Method: (1) sort the routes cheapest first; (2) take the next route; if its two ends are already joined through routes already chosen, reject it (it would close a loop), otherwise accept it; (3) stop when every route has been checked. Keep a list of the groups of connected branches and update it after every accepted route.

2. Routes sorted cheapest first, and the decision on each

#	Route	Min	Decision	Groups after this step	Total	Reason
1	HUB-KAR	13	ACCEPT	{HUB,KAR} + QUS,UMS,JAF,SHJ,AJM	13	joins two groups
2	HUB-QUS	16	ACCEPT	{HUB,KAR,QUS} + UMS,JAF,SHJ,AJM	29	joins two groups
3	KAR-UMS	20	ACCEPT	{HUB,KAR,QUS,UMS} + JAF,SHJ,AJM	49	joins two groups
4	HUB-UMS	23	REJECT	{HUB,KAR,QUS,UMS} + JAF,SHJ,AJM	49	loop, HUB and UMS already joined via HUB - KAR - UMS
5	HUB-SHJ	24	ACCEPT	{HUB,KAR,QUS,UMS,SHJ} + JAF,AJM	73	joins two groups
6	UMS-JAF	24	ACCEPT	{HUB,KAR,QUS,UMS,JAF,SHJ} + AJM	97	joins two groups
7	KAR-QUS	25	REJECT	{HUB,KAR,QUS,UMS,JAF,SHJ} + AJM	97	loop, KAR and QUS already joined via KAR - HUB - QUS
8	SHJ-AJM	27	ACCEPT	{HUB,KAR,QUS,UMS,JAF,SHJ,AJM} ALL JOINED	124	joins two groups
9	QUS-SHJ	29	REJECT	{HUB,KAR,QUS,UMS,JAF,SHJ,AJM} ALL JOINED	124	loop, QUS and SHJ already joined via QUS - HUB - SHJ
10	KAR-SHJ	30	REJECT	{HUB,KAR,QUS,UMS,JAF,SHJ,AJM} ALL JOINED	124	loop, KAR and SHJ already joined via KAR - HUB - SHJ
11	QUS-UMS	32	REJECT	{HUB,KAR,QUS,UMS,JAF,SHJ,AJM} ALL JOINED	124	loop, QUS and UMS already joined via QUS - HUB - KAR - UMS
12	KAR-JAF	34	REJECT	{HUB,KAR,QUS,UMS,JAF,SHJ,AJM} ALL JOINED	124	loop, KAR and JAF already joined via KAR - UMS - JAF
13	QUS-AJM	37	REJECT	{HUB,KAR,QUS,UMS,JAF,SHJ,AJM} ALL JOINED	124	loop, QUS and AJM already joined via QUS - HUB - SHJ - AJM

Start: seven separate groups {HUB} {KAR} {QUS} {UMS} {JAF} {SHJ} {AJM}. Two routes tie at 24 min (HUB-SHJ and UMS-JAF); they join different groups, so the order between them does not matter.

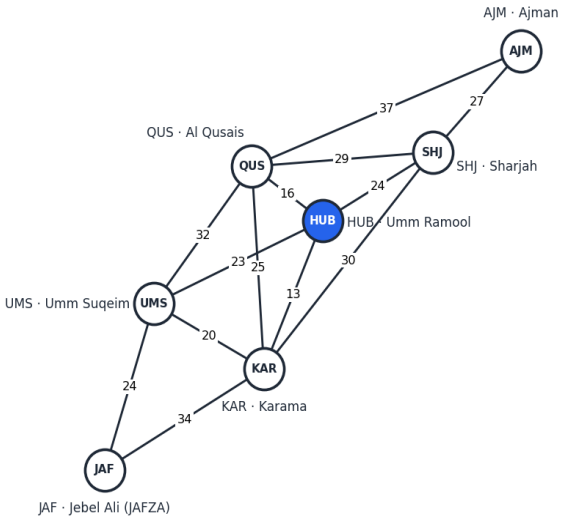
3. Result

Minimum network	Minutes
HUB-KAR	13
HUB-QUS	16
KAR-UMS	20
HUB-SHJ	24
UMS-JAF	24
SHJ-AJM	27
Total (6 routes)	**124**

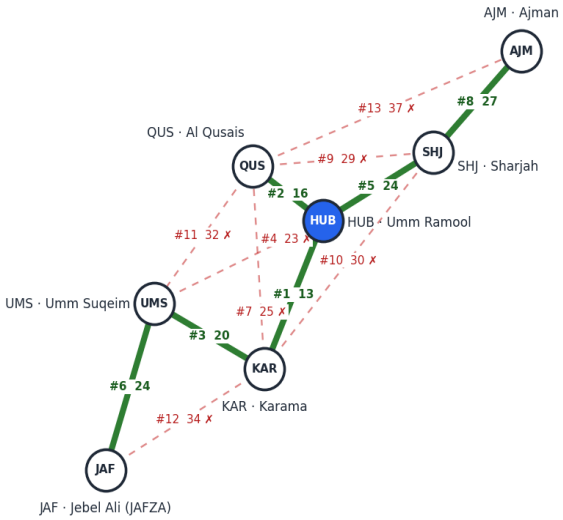
All 13 routes: 334 min. Minimum network: 124 min. Saved: 334 − 124 = 210 min = 62.9% (210 ÷ 334). Routes kept: 7 branches − 1 = 6. Routes rejected: 7, all because they would form a loop.

4. The picture

All routes under X = 38 min · 13 routes · 334 min in total



Kruskal's (#n = order tried, ✗ = loop) · 6 routes · 124 min



Times: Google Maps driving, depart Thu 5 Nov 2026 07:00, midpoint of typical range. Each pair used in both directions. Layout is schematic (real W-E / S-N positions, not to scale).

Minimum network: HUB-KAR 13 + HUB-QUS 16 + KAR-UMS 20 + HUB-SHJ 24 + UMS-JAF 24 + SHJ-AJM 27 = 124 min · All 13 routes = 334 min · Saved 210 min = 62.9%

5. Cross-checks

Prim's from HUB adds the cheapest route leaving the tree each time: HUB-KAR (13), HUB-QUS (16), KAR-UMS (20), HUB-SHJ (24), UMS-JAF (24), SHJ-AJM (27). Total 124 min: same 6 routes, same total as Kruskal's. networkx nx.minimum_spanning_tree also gives 124 min. Python check: python kruskal_check.py.

6. Questions I expect, and my answers

Question	Answer
Which routes did Kruskal's reject, and why?	HUB-UMS (23), KAR-QUS (25), QUS-SHJ (29), KAR-SHJ (30), QUS-UMS (32), KAR-JAF (34), QUS-AJM (37). Each one's two ends were already joined by cheaper routes, so keeping it would only add a loop and cost.
Would Prim's give the same answer?	Yes: same 6 routes, 124 min. Kruskal's sorts all routes and merges groups; Prim's grows one tree from a start branch. The only tie (24 = 24) does not change the result.
Why exactly 6 routes?	7 branches need at least 6 routes to be connected. A 7th would close a loop. Fewer than 6 would leave a branch cut off.
Why not run all 13?	334 min against 124 min: 62.9% saved. This treats drive minutes as a stand-in for running cost, which is a simplification.
Why is the minimum network not the fastest delivery route? (Routing)	It minimises total network cost, not hub-to-branch time. HUB→UMS: 33 min on the minimum network vs 23 min fastest; HUB→JAF: 57 min on the minimum network vs 47 min fastest. Tab 2 (Dijkstra) uses all 13 routes.
What is the weakness of the minimum network?	A tree has no spare routes. Branches QUS, JAF and AJM hang on a single route, and if HUB, KAR, UMS or SHJ is closed the network splits. The full 13-route network has no single point of failure (Data Lead). Cheapest and most robust pull in opposite directions.
What if X is lowered by 5 (to 33)?	11 routes remain (263 min). The minimum network is unchanged: same 6 routes, 124 min, because every kept route is 27 min or less. The saving falls to 52.9%.
At what X does Kruskal's change?	Only at X of 27 or less: SHJ-AJM (27) disappears and Ajman is cut off, so Kruskal's returns a forest, not a tree (5 routes at X = 27).

Data: Google Maps driving times, depart Thu 5 Nov 2026 07:00, midpoint of typical range, one time per pair used in both directions (Data Lead).